UC San Diego

DSC 102
Systems for Scalable Analytics

Alexander Kolovos

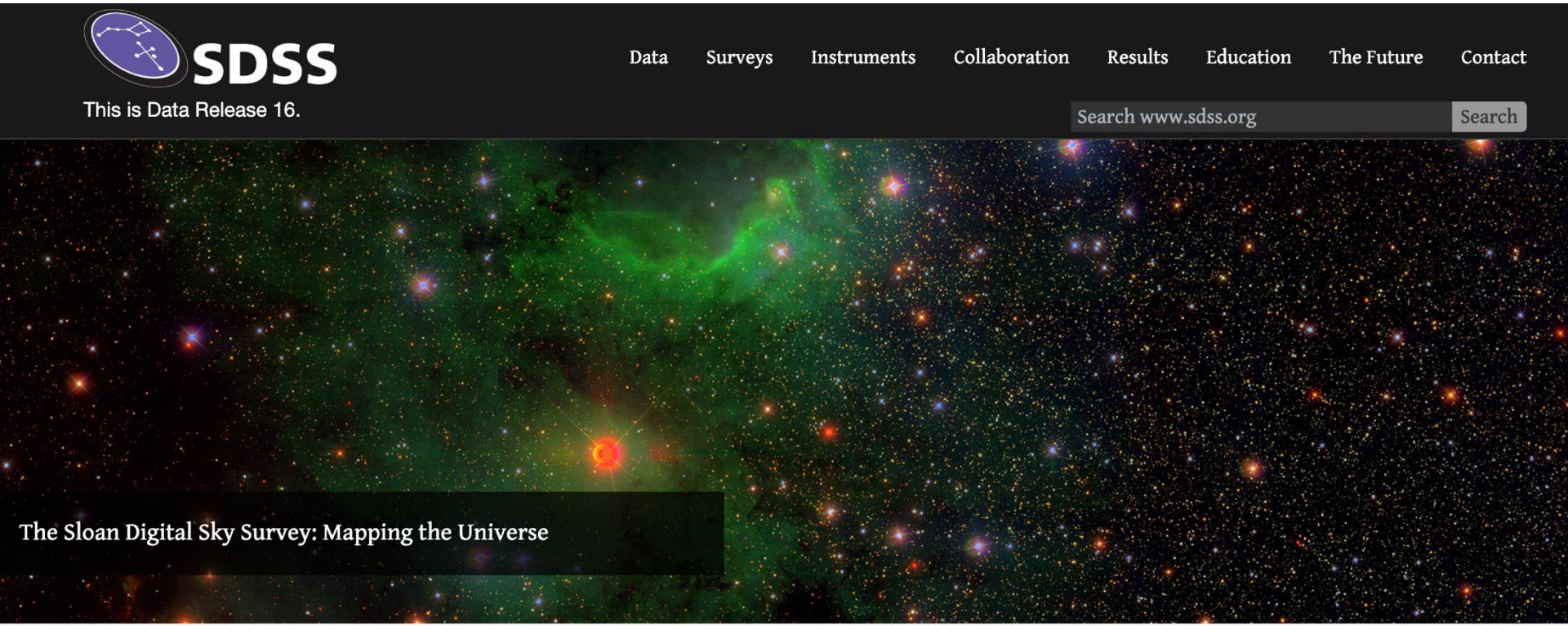
Topic 2: Parallel and Scalable Data Processing

Part 1: Parallelism Basics

Ch. 9.4, 12.2, 14.1.1, 14.6, 22.1-22.3, 22.4.1, 22.8 of Cow Book

Q: *Why bother with large-scale data? Why does sampling not suffice?*

Large-Scale Data in Astronomy

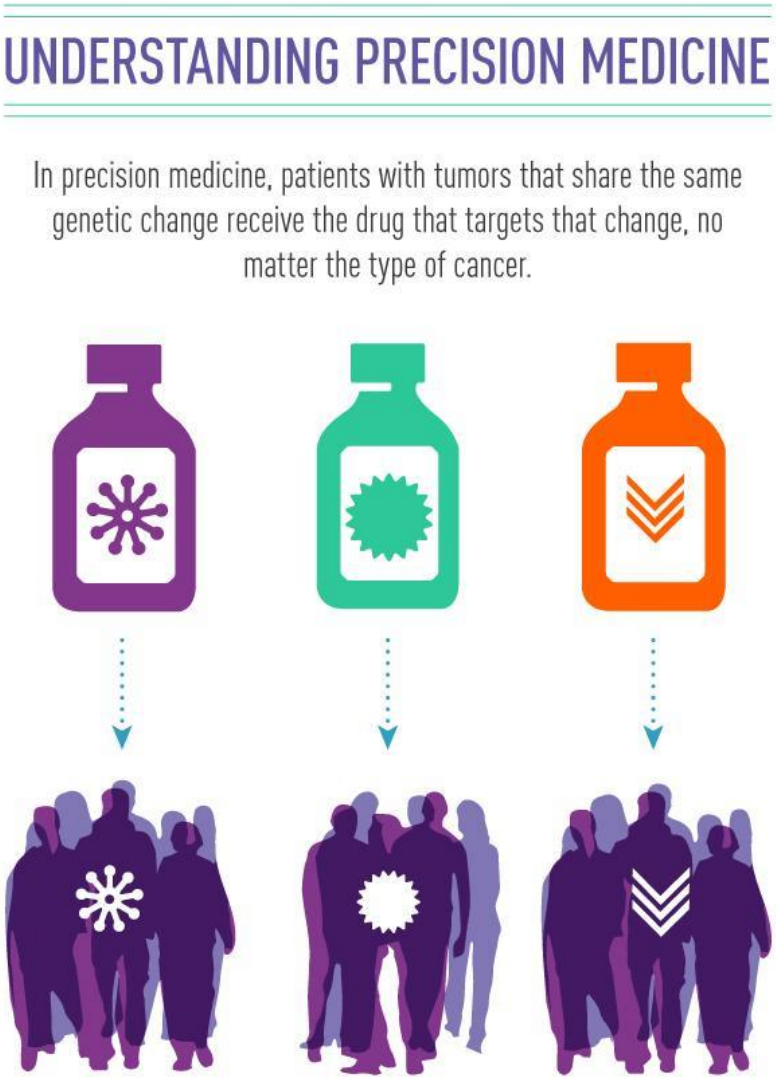


The Sloan Digital Sky Survey has created the most detailed three-dimensional maps of the Universe ever made, with deep multi-color images of one third of the sky, and spectra for more than three million astronomical objects. Learn and explore all phases and surveys—past, present, and future—of the SDSS.

High-res. images: ~200 GB per day since 2000 (1PB+)

Astronomers can study complex galactic evolution behaviors

Large-Scale Data in Genomics



Precision Medicine is becoming a reality

Analyze genomes across cohorts and prescribe targeted drugs and treatments

~3GB genome per human

~1EB for USA

Large-Scale Data in E-commerce

Log all user behavior (views, clicks, pauses, searches, etc.)

Recommender systems combine TBs of data from all users and movies to deliver a tailored experience

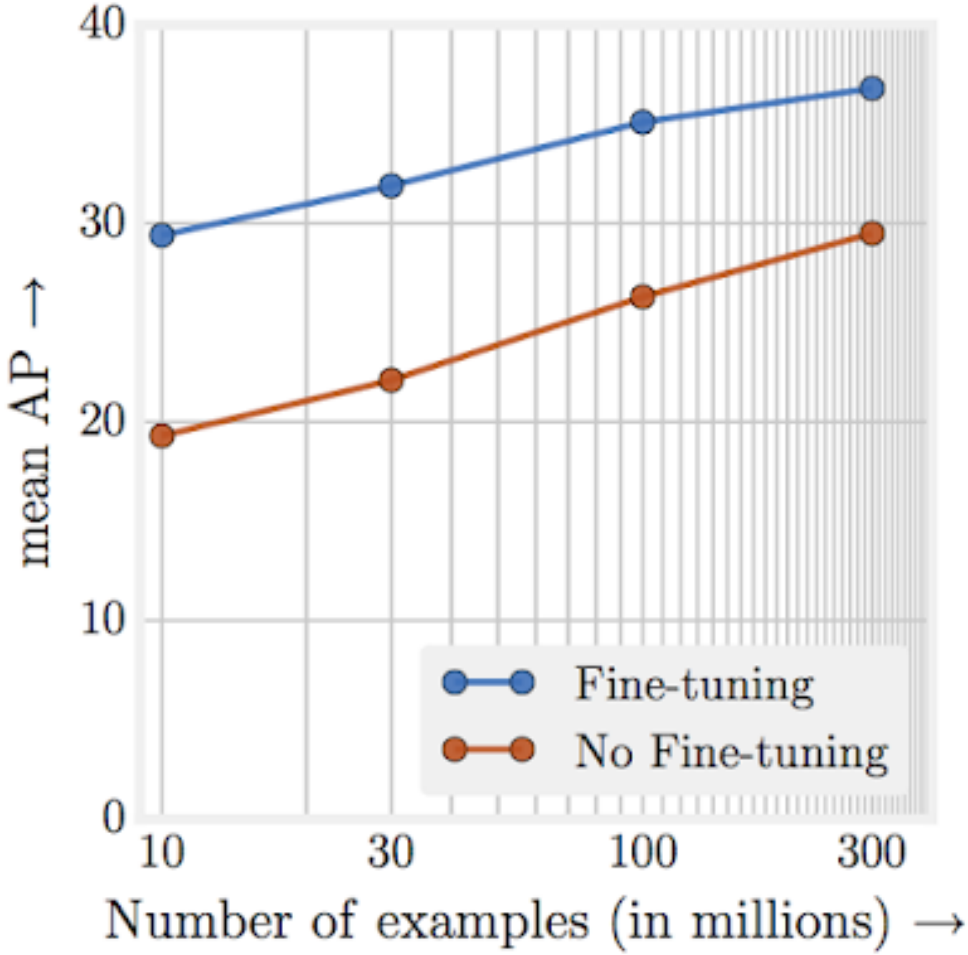
Large-Scale Data in Computer Vision

10million+ images labeled (20,000 classes) by crowdsourcing

>500GB uncompressed as tensors

Harbinger of deep learning revolution

“The Unreasonable Effectiveness of Data”



When **prediction target complexity** is high:

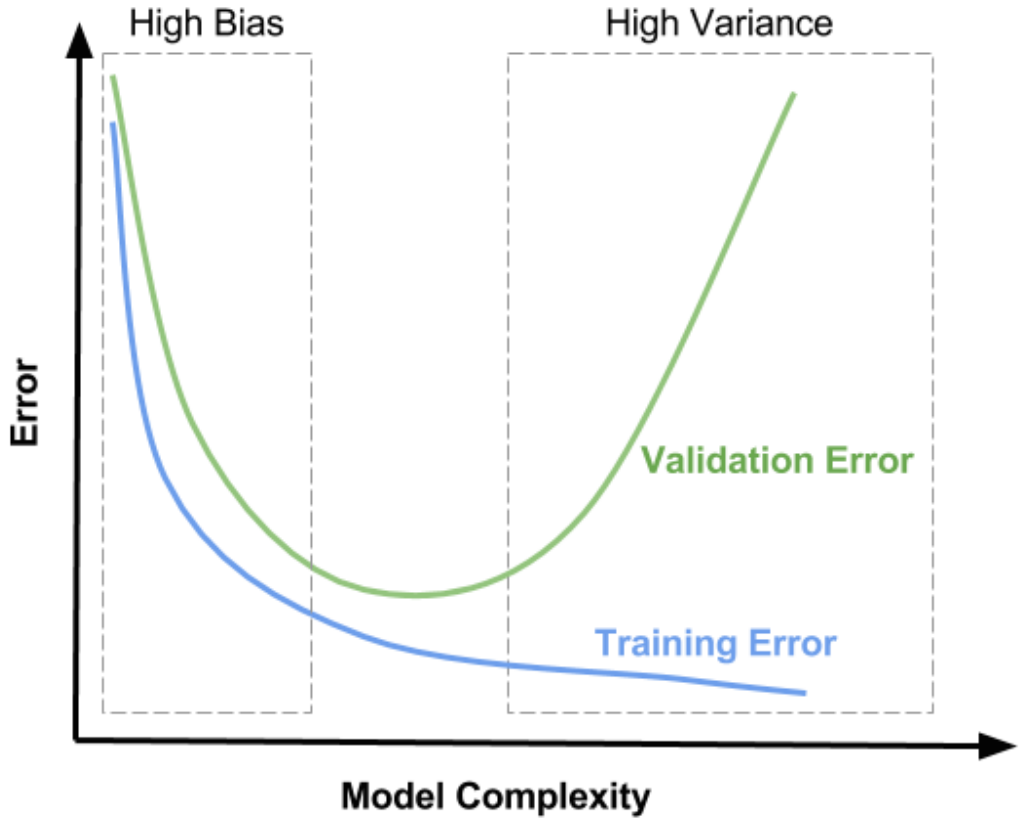
- more training data coupled with
- more complex models

⇒ higher accuracy as number of training examples grows

Bonus remark: What's the difference?...
Last sentence refers to *accuracy*
Figure refers to *precision*.
Precision: How close measurements are to each other (relates to random errors).
Accuracy: Closeness to **true** value (relates to systematic errors).

https://ai.googleblog.com/2017/07/revisiting-unreasonable-effectiveness.html

Bias-Variance Tradeoff of ML



High **Bias**: Roughly, model is not rich enough to represent data

High **Variance**: Model *overfits* to given data; poor *generalization*

Large-scale training data lowers variance and raises accuracy!

Why Large-Scale Data?

- Large-scale data is a game changer in data science:
  - Enables **study of granular phenomena** in sciences, businesses, etc. not possible before
  - Enables **new applications** and personalization/customization
  - Enables more **complex ML prediction targets** and mitigates variance to offer **high accuracy**
- Hardware has kept pace to power the above:
  - Storage capacity has exploded (PB clusters)
  - Compute capacity has grown (multi-core, GPUs, etc.)
  - DRAM capacity has grown (10GBs to TBs)
  - Cloud computing is “democratizing” access to hardware; SaaS

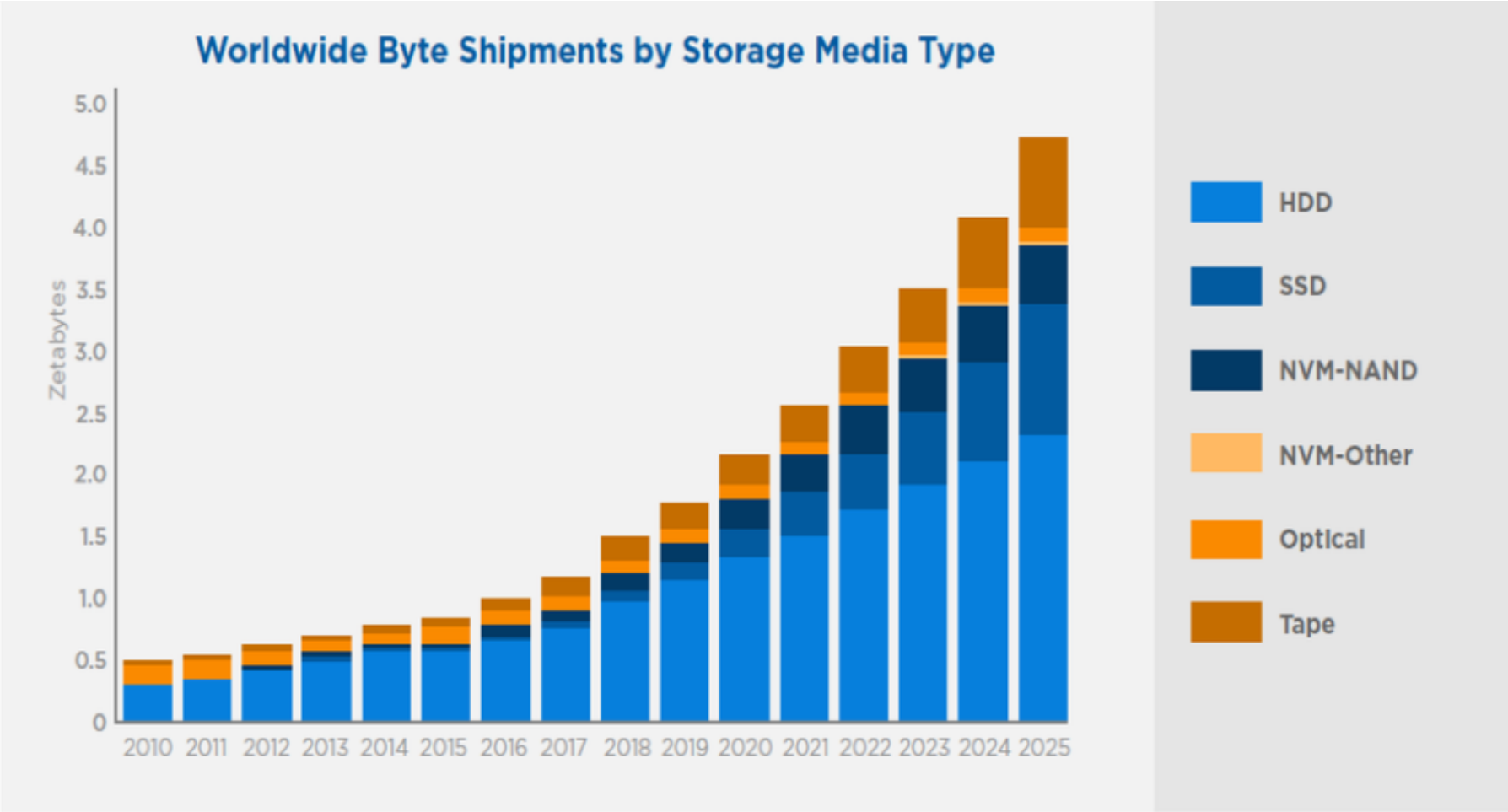
“Big Data”

- Marketing term; think “Big” as in “Big Oil” or “Big Government” or “Big Tech”, not “big building”
  - Became popular in late 2000s to early 2010s
  - Wikipedia says: “Data that is so large and complex that existing toolkits [read RDBMSs!] are not adequate”
- Typical characterization by 3 Vs:
  - **Volume**: larger than single-node DRAM
  - **Variety**: relations, docs, tweets, multimedia, etc.
  - **Velocity**: high generation rate, e.g., sensors, surveillance

Why “Big Data” now? 1. Applications

- New “data-driven mentality” in almost all human endeavors:
- **Web**: search, e-commerce, e-mails, social media
- **Science**: satellite imagery, CERN’s LHC, document corpora
- **Medicine**: pharmacogenomics, precision medicine
- **Logistics**: sensors, GPS, “Internet of Things”
- **Finance**: high-throughput trading, monitoring
- **Humanities**: digitized books/literature, social media
- **Governance**: e-voting, targeted campaigns, NSA
- ⑤

Why “Big Data” now? 2. Storage



Source: Data Age 2025, sponsored by Seagate with data from IDC Global DataSphere, Nov 2018

To analyze large-scale data, parallel and scalable data systems are indispensable!

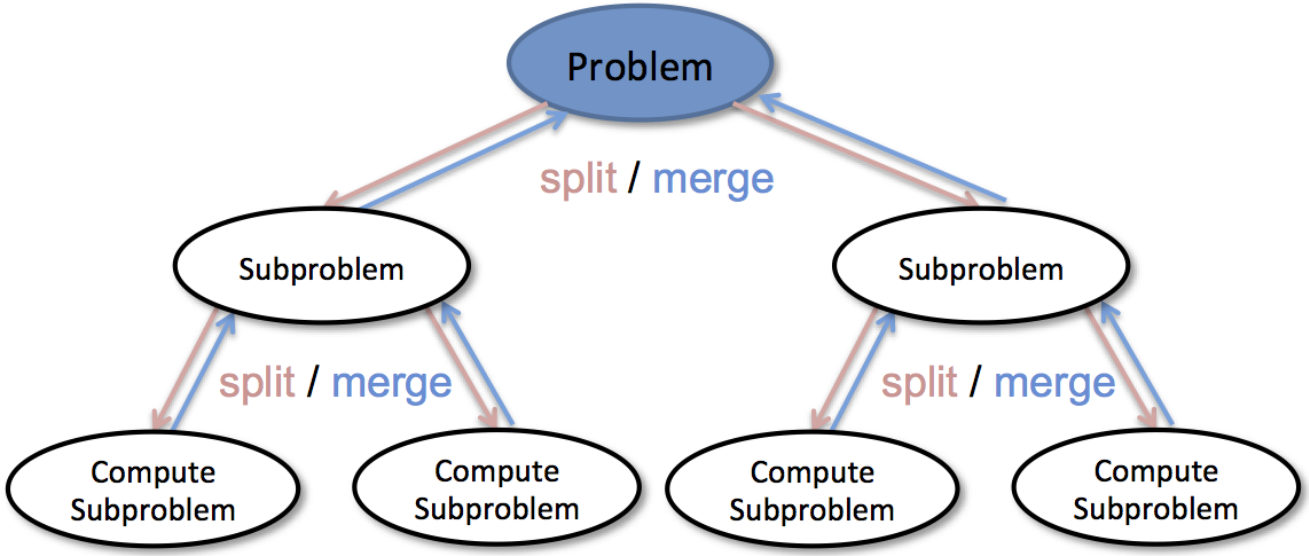
Topic 2 – Outline

- Basics of Parallelism
  - Task Parallelism; Dask
  - Single-Node Multi-Core; SIMD; Accelerators
- Basics of Scalable Data Access
  - Paged Access; I/O Costs; Layouts/Access Patterns
  - Scaling Data Science Operations
- Data Parallelism: Parallelism + Scalability
  - Data-Parallel Data Science Operations
  - Optimizations and Hybrid Parallelism

Parallel Data Processing

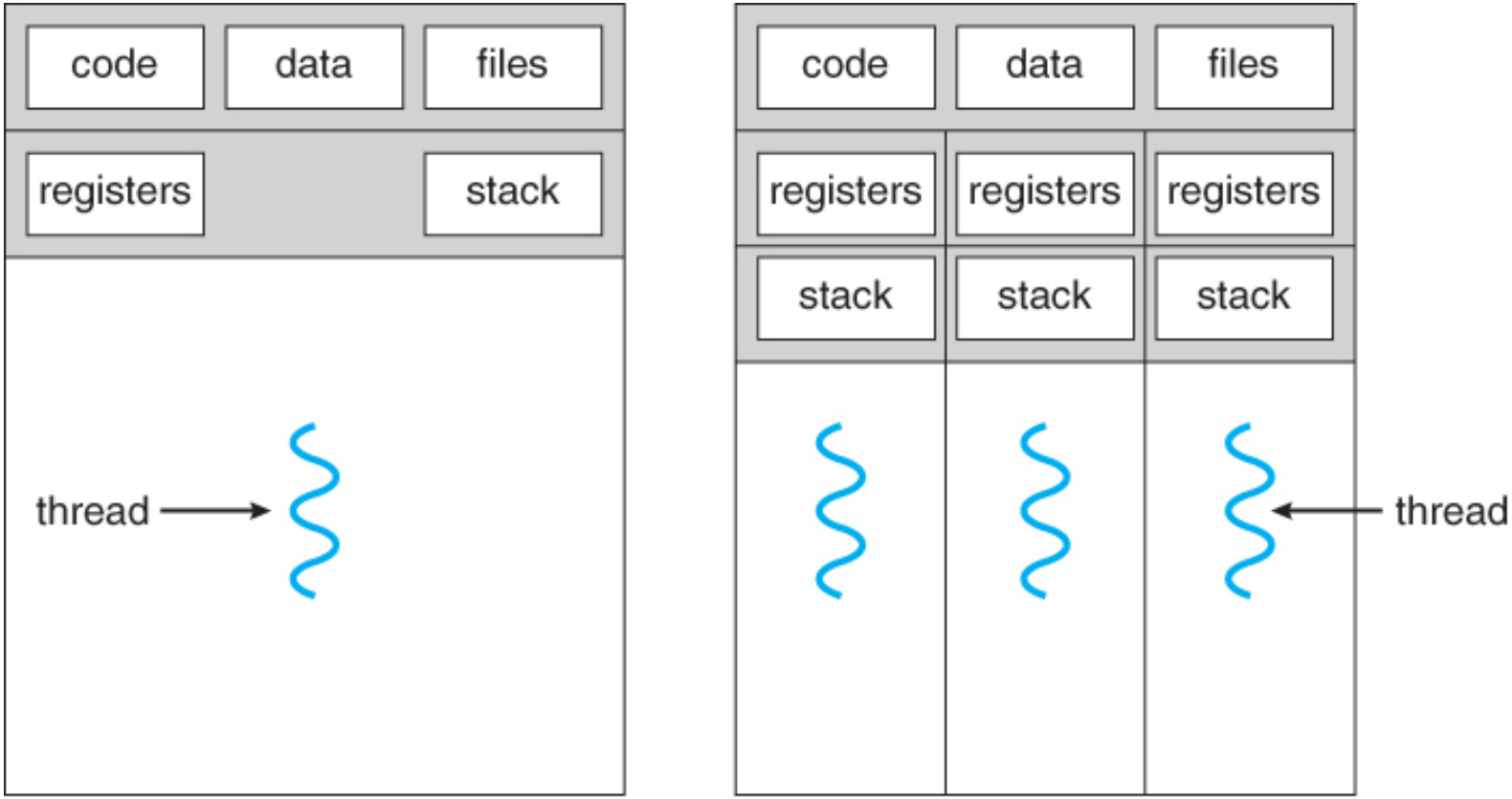
Central Issue: Workload takes too long for one processor!

Basic Idea: Split up workload across processors and perhaps also across machines/workers (aka "Divide and Conquer")



https://medium.com/cracking-the-data-science-interview/divide-and-conquer-algorithms-b135681d08fc

Multiple Threads in a Process



single-threaded process

multithreaded process

Concurrent Or Parallel?

	Concurrency	Parallelism
Definition	Dealing with multiple tasks by interleaving their execution	Executing multiple tasks simultaneously
Core idea	Structure	Execution
Hardware required	Single or multi-core	Multi-core / multiple processors
Tasks run at same instant?	No — tasks take turns	Yes — tasks run at the same time
Primary goal	Responsiveness, progress on multiple tasks	Speed, throughput
Example	A single barista taking orders while brewing coffee	Multiple baristas each independently making drinks
Code example	`async/await`, coroutines, event loops	Threads, `multiprocessing`, SIMD
Bottleneck	I/O-bound tasks	CPU-bound tasks
Failure isolation	Shared state bugs, race conditions	Same, plus inter-process coordination overhead
Rob Pike's framing	"Composition of independently executing processes"	"Simultaneous execution of (possibly related) computations"

The key insight: concurrency is about *design* — how you structure a program to handle many things. Parallelism is about *execution* — actually doing many things at once.

You can have concurrency without parallelism (a single-core scheduler), parallelism without concurrency (a SIMD instruction), or both together (a multi-threaded web server).

Execution Patterns Of Parallelism

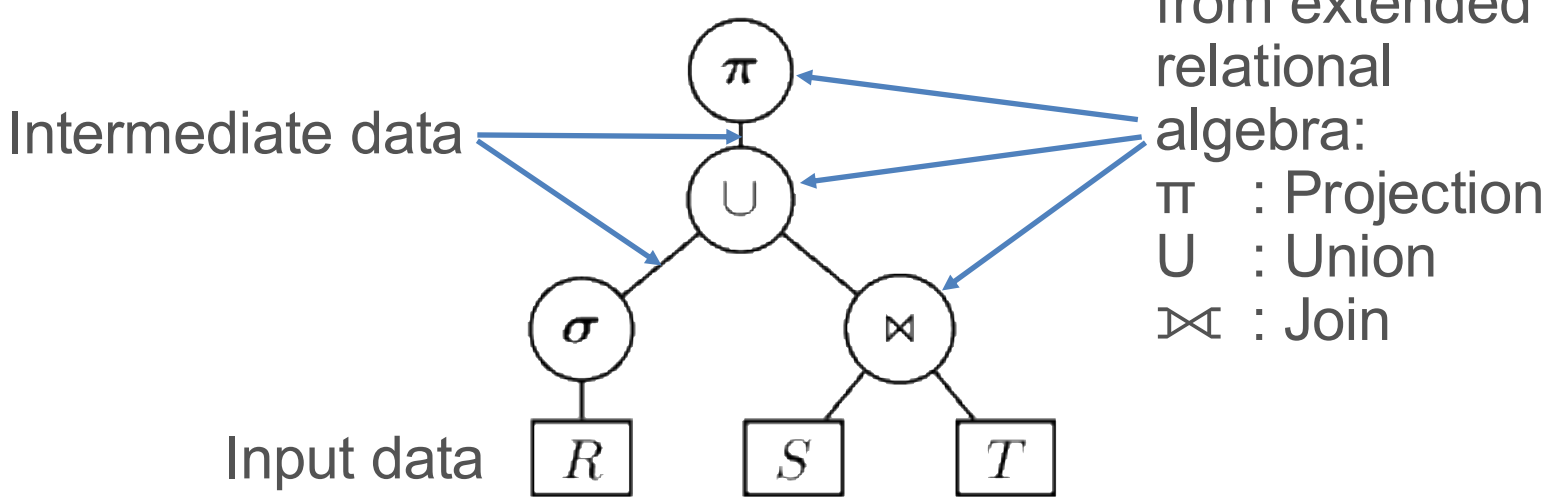
		Code	
		Single Program	Multiple Programs
Data	Non-partitioned	Sequential program on a single data source	Replicated Data => MISD E.g., DASK
	Partitioned	SPMD, SIMD Map Reduce	MIMD E.g.,Task Graphs, Spark

New Parallelism Concept: Dataflow

- Common in parallel data processing: **"Dataflow Graph"**:
  - A *directed graph* representation of a program with vertices being *abstract operations* from a restricted set of computational primitives:
    - Extended relational dataflows: RDBMS, Pandas, Modin
    - Matrix/tensor dataflows: NumPy, PyTorch, TensorFlow
- Enables us to reason about data-intensive programs at a higher level (logical level?)
- **Task Graph**: Similar but coarse-grained; vertex is a process

Example: Relational Dataflow Graph

$$\pi(\sigma(R) \cup S \bowtie T)$$

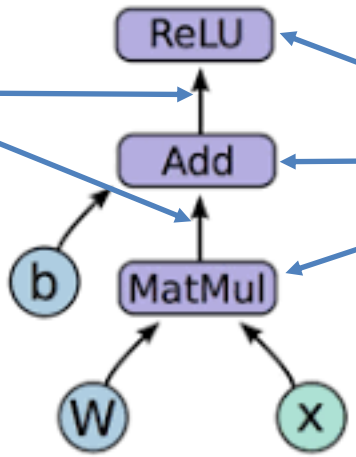


Aka **Logical Query Plan** in the DB systems world

Example: Tensor Dataflow Graph

$$ReLU(WX + b)$$

Intermediate data

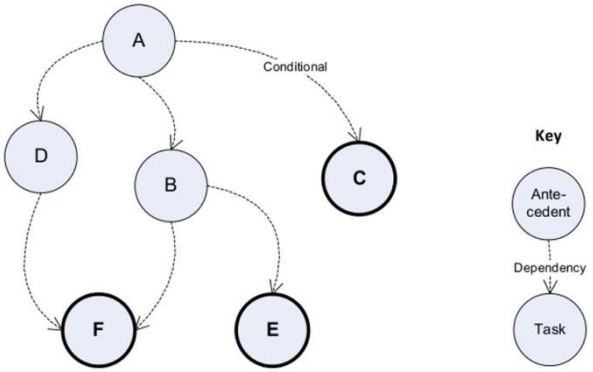


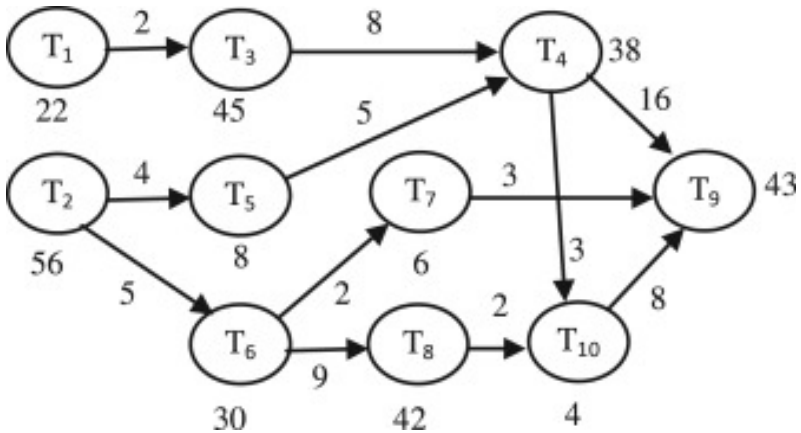
Input data

Operators
from LA/DL tool's
tensor algebra
MatMul : Matrix
multiplication
ReLU : Activation
function

Aka **Neural Computational Graph** in the ML systems world

Example: Task Graph





- More coarse-grained than operator-level dataflows
- Vertex: A full task/process
- Edge: A dependency between tasks
- Directed Acyclic Graph model (DAG) common
- Data may not be shown

Dask conflates the concepts of Dataflow and Task graphs because an "operation" on a Dask DataFrame becomes its own separate process/program under the hood!

https://docs.dask.org/en/latest/graphviz.html

Parallel Data Processing

Central Issue: Workload takes too long for one processor!

Basic Idea: Split up workload across processors and perhaps also across machines/workers (aka "Divide and Conquer")

Key parallelism paradigms in data systems:

Dataset is:	**Shared**	**Replicated**	**Partitioned**
Within a node:	"SIMD" "Pipelining"	"Task Parallel" Systems	"Data Parallel" Systems
Across nodes:	N/A	DASK	APACHE Spark

Note: SIMD = Single Instruction, Multiple Data

Topic 2 – Outline

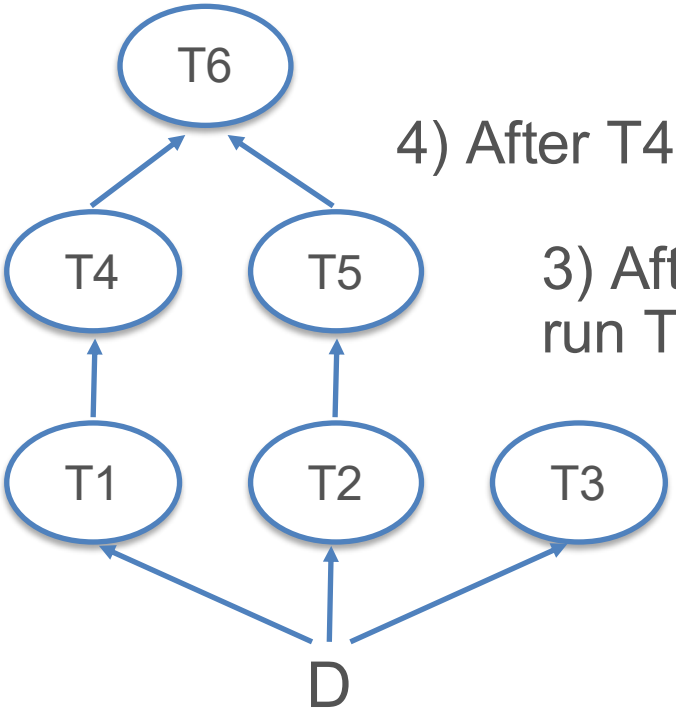
- Basics of Parallelism
  - Task Parallelism; Dask
  - Single-Node Multi-Core; SIMD; Accelerators
- Basics of Scalable Data Access
  - Paged Access; I/O Costs; Layouts/Access Patterns
  - Scaling Data Science Operations
- Data Parallelism: Parallelism + Scalability
  - Data-Parallel Data Science Operations
  - Optimizations and Hybrid Parallelism

Task Parallelism

Basic Idea: Split up *tasks* across workers; if there is a common dataset that they read, just make copies of it (aka *replication*)

Example: Given 3 workers

This is your PA1 setup! Except, Dask Scheduler puts tasks on workers for you.



4) After T4 & T5 end, run T6 on, say, W1; W2 is *idle*

3) After T1 ends, run T4 on W1; after T2 ends, run T5 on W2; after T3 ends, W3 is *idle*

2) Put T1 on worker 1 (W1), T2 on W2, T3 on W3; run all 3 in parallel

1) Copy whole D to all workers

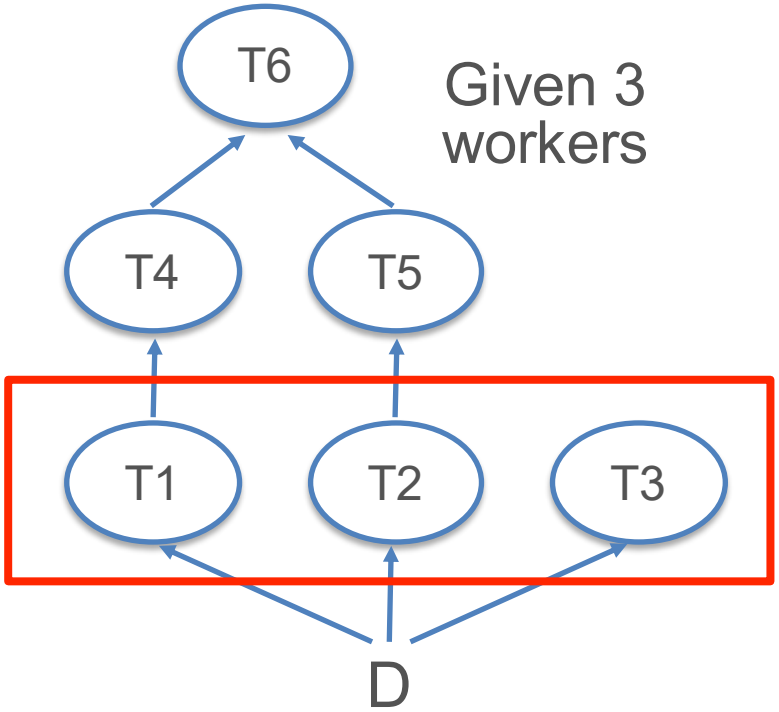
Task Parallelism

- **Topological sort** of tasks in task graph for scheduling
- Notion of a “worker” can be at processor/core level, not just at node/server level
  - *Thread-level* parallelism possible instead of process-level
  - E.g., Dask: 4 worker nodes x 4 cores = 16 workers total
- **Main pros** of task parallelism:
  - **Simple** to understand; easy to implement
  - **Independence** of workers => low software complexity
- **Main cons** of task parallelism:
  - Data replication across nodes; **wastes memory**/storage
  - **Idle times** possible on workers

Degree of Parallelism

- Degree of Parallelism: The largest amount of *concurrency* possible in the task graph, i.e., how many tasks can be run simultaneously.

Example:



Q: *How do we quantify the runtime performance benefits of task parallelism?*

In this example we observe:

- Over time, it happens that fewer workers needed for parallel tasks (degree of parallelism for later segments keeps dropping).
- Degree of parallelism is 3 (more than 3 workers is not useful for this workload!)

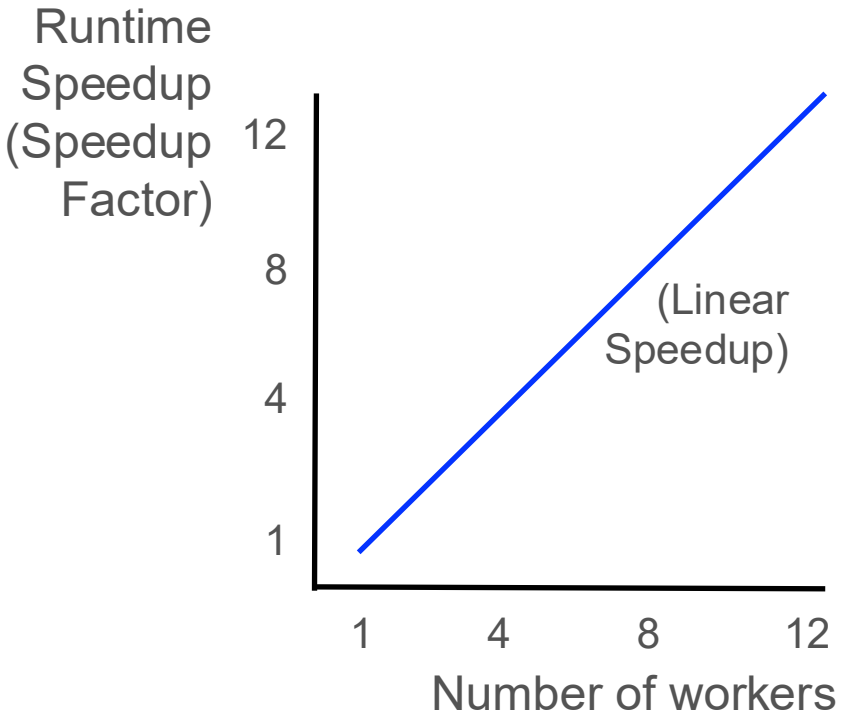
Quantifying Parallelism Benefit: Speedup

$$\textbf{Speedup} = \frac{\text{Completion time given only 1 worker}}{\text{Completion time given n (>1) workers}}$$

Q: *Given n workers,*
can we get a speedup of n?

It depends!

(...on degree of parallelism,
task dependency graph structure,
intermediate data sizes, etc.)

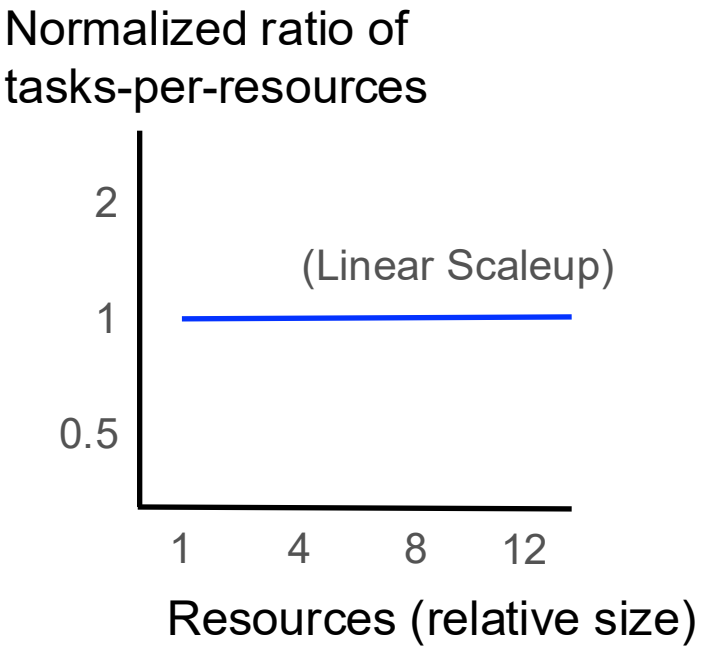


Quantifying Parallelism Benefit: Scaleup

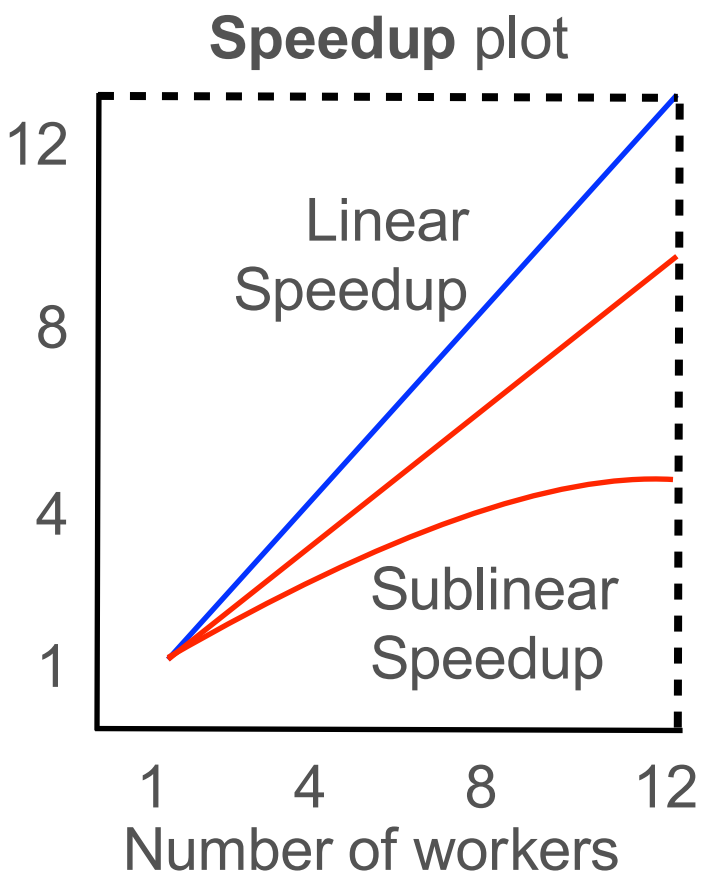
Scaleup : The ability of a system
to retain the same performance ratio of tasks-per-resources
when both the tasks and the resources increase at same rate.

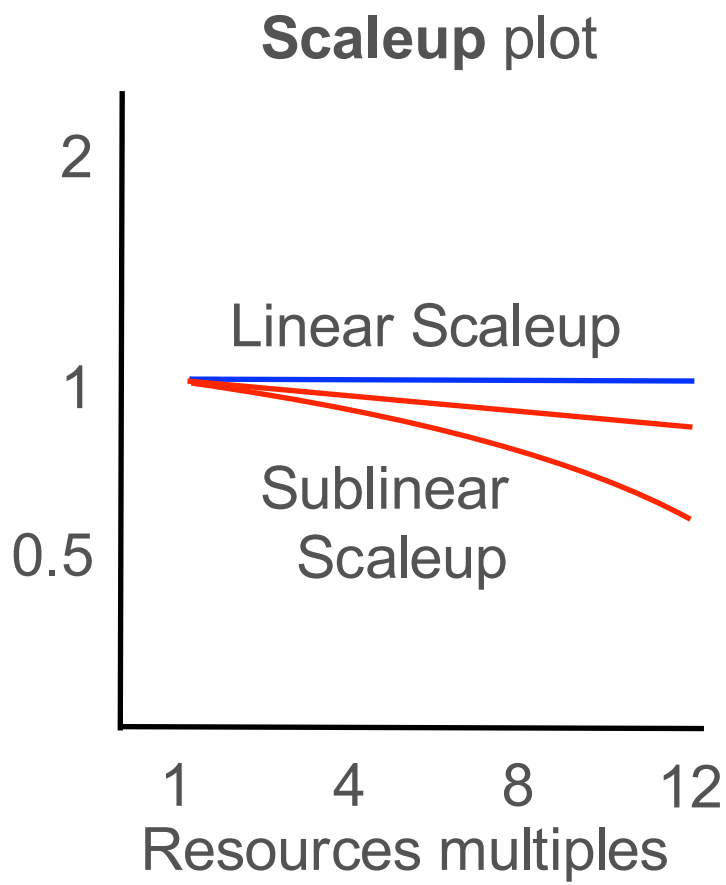
In the above:

- "Task" can refer to a single or series of computations, queries, etc.
- "Resources" can refer to # of workers, DRAM, storage size, etc.
- "Increase" refers to using multiple instances of an initial task and initial set of resources.



Quantifying Parallelism Benefit





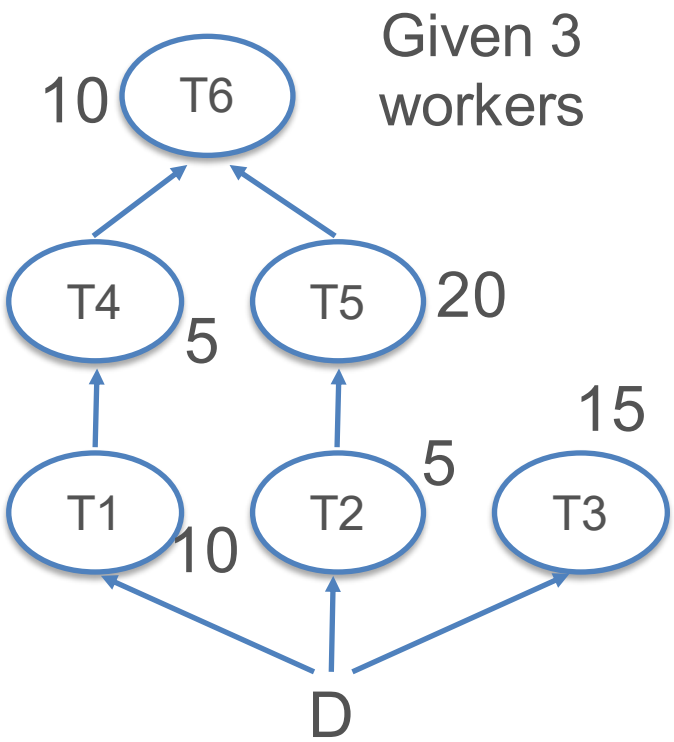
Most commonly, scaling does not demonstrate ideal linear behavior.

***Q:** Is <u>superlinear</u> speedup/scaleup ever possible?*

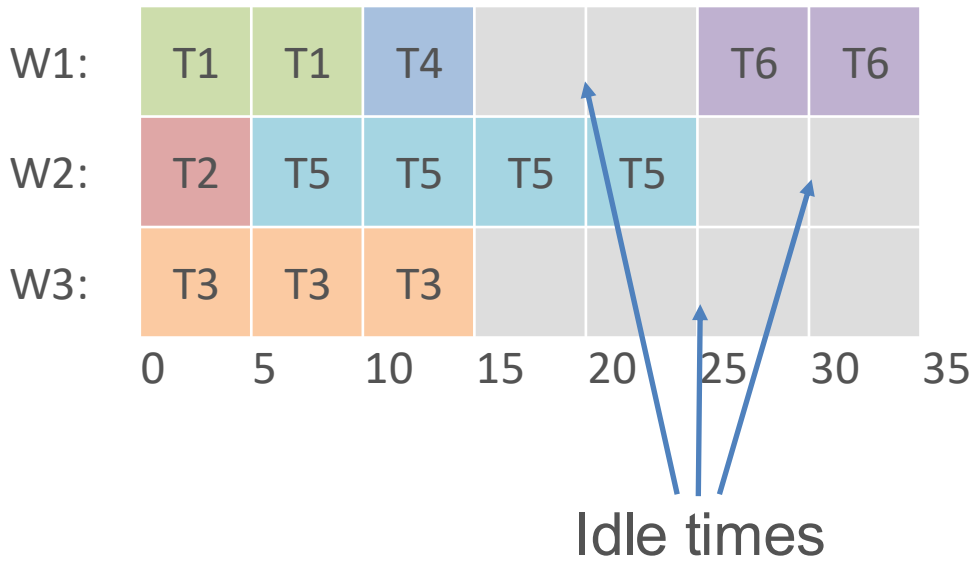
Idle Times in Task Parallelism

- Due to varying task completion times and varying degrees of parallelism in workload, idle workers waste resources

Example:



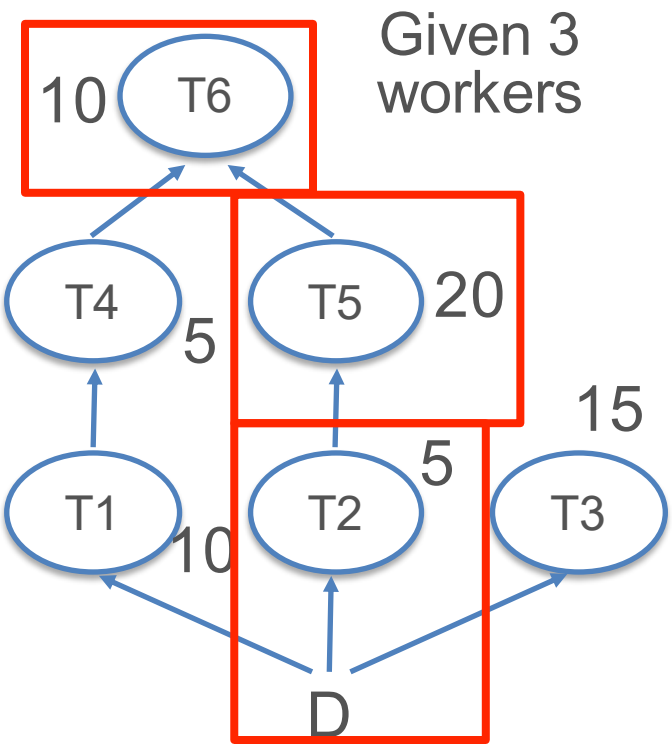
Gantt Chart visualization of schedule:



Idle Times in Task Parallelism

- Due to varying task completion times and varying degrees of parallelism in workload, idle workers waste resources

Example:

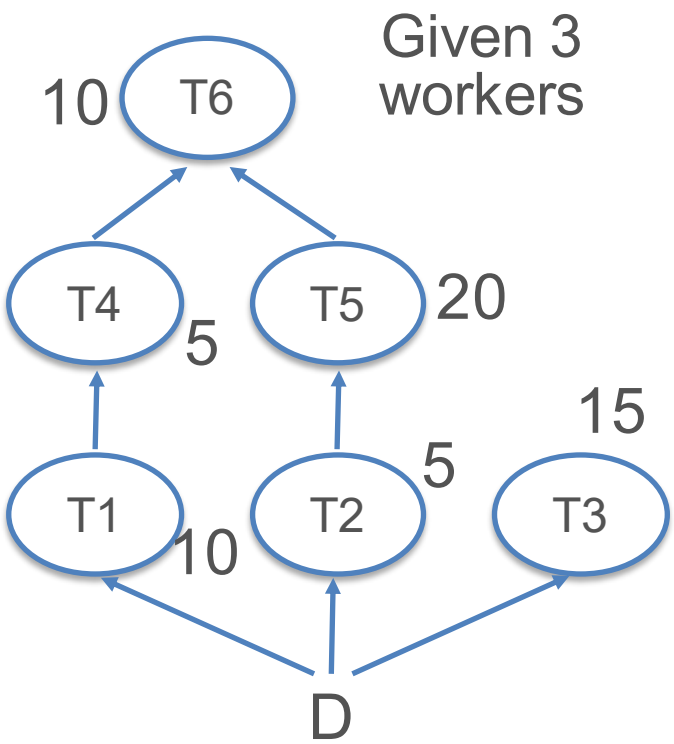


- In general, overall workload's completion time on task-parallel setup is always *lower bounded* by the **longest path** in the task graph
- Possibility: A task-parallel scheduler can "release" a worker if it knows that will be idle till the end
  - Can saves costs in cloud

Calculating Task Parallelism Speedup

- Due to varying task completion times and varying degrees of parallelism in workload, idle workers waste resources

Example:



Completion time with 1 worker:

10+5+15+5+20+10 = 65

Parallel completion time:

(Longest path on task graph) 35

Speedup = 65/35 = 1.9x

Ideal/linear speedup is 3x

Q: *Why is it only 1.9x?*

Efficiency

- The efficiency of an algorithm using P processors is

$$\textbf{Efficiency} = \frac{\text{Speedup}(P)}{P}$$

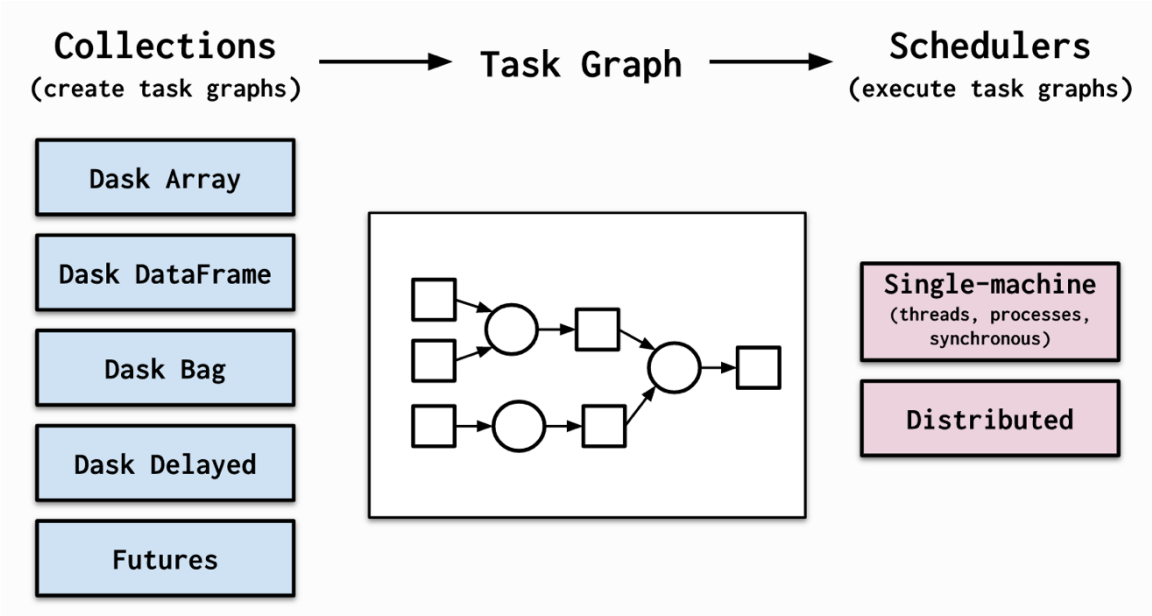
- Efficiency estimates how well-utilized the processors are in running the parallel program.
- Ideal speedup means Efficiency = 1 (100% efficiency).

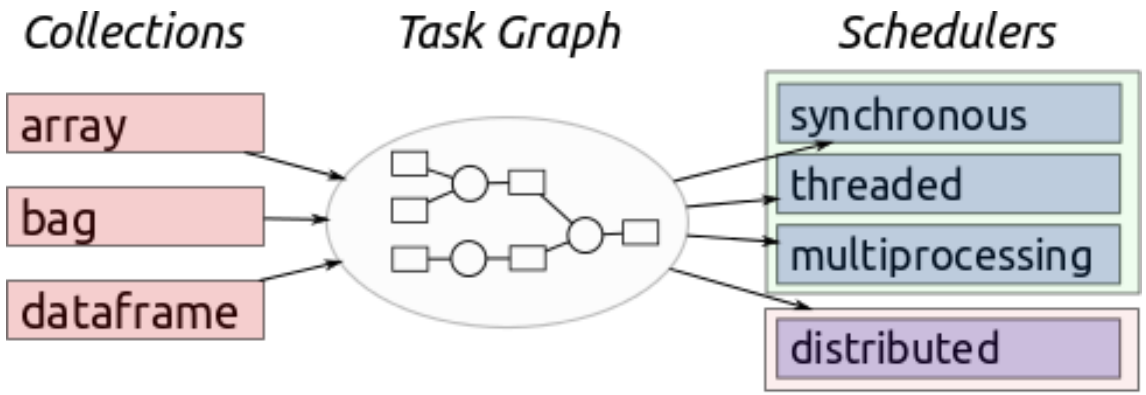
Task Parallelism in Dask

- *"Dask is a flexible library for **parallel** computing in Python"*
- **2 key components:**
  - APIs for data science ops on large data
  - Dynamic task scheduling on multi-core/multi-node
- **Design desirables:**
  - *Pythonic*: Stay within PyData stack (e.g., no JVM)
  - *Familiarity*: Retain APIs of NumPy, Pandas, etc.
  - *Scaling Up*: Seamlessly exploit all cores
  - *Scaling Out*: Easily exploit cluster (needs setup)
  - *Flexibility*: Can schedule custom tasks too
  - *Fast?*: "Optimized" implementations under APIs

https://docs.dask.org/en/latest/

Task Parallelism in Dask





https://docs.dask.org/en/latest/
https://docs.dask.org/en/latest/scheduling.html

Dask's Workflow

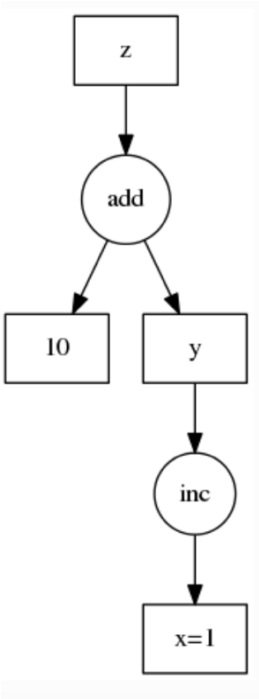
- **"Lazy Evaluation":**
  - Ops on data structures are NOT executed immediately
  - Triggered manually, e.g., *compute()*
  - Dataflow graph / task graph is built under the hood

```python
def inc(i):
    return i + 1

def add(a, b):
    return a + b

x = 1
y = inc(x)
z = add(y, 10)
```

User code using their API

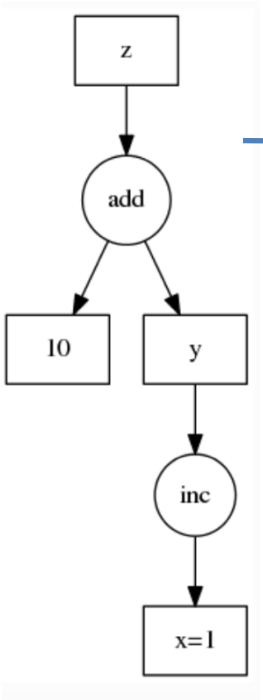
```python
d = {'x': 1,
     'y': (inc, 'x'),
     'z': (add, 'y', 10)}
```

Internal dictionary with key-value pair representation of dataflow/task graph



Dask's Workflow

- Rest of the Dask's workflow for distributing computations:



"Optimize" Task Graph

Serialize Task Graph (dictionary)

Communicate it to Scheduler

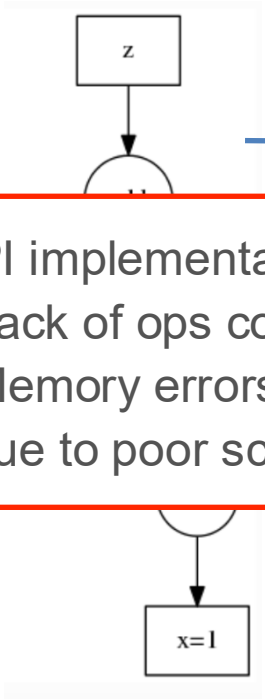
Scheduler divvies up work for cores/nodes

Each worker does its job and notifies Scheduler

https://docs.dask.org/en/latest/phases-of-computation.html

Possible Bottlenecks/Issues in Dask

- Rest of the Dask's workflow for distributing computations:



API implementation:
- Lack of ops coverage
- Memory errors in ops due to poor scalability

"Optimize" Task Graph

Poor/missed optimizations

Serialize Task Graph (dictionary)

Huge task graphs take too long to serialize and/or communicate

Communicate to Scheduler

Scheduler divvies up work for cores/nodes

Poor scalability of Scheduler

Each worker does its job and notifies Scheduler

Machine/network failures

Dask: Task-Parallelism Best Practices

- Is Dask even needed? Will single-node in-memory tool suffice?
- **Data Partition sizes:**
  - Avoid too few chunks (low degree of par.)
  - Avoid too many chunks (task graph overhead)
  - Be mindful of available DRAM
  - Rough guidelines they give:
    - # data chunks ~ 3x-10x # cores, but
    - # cores x chunk size must be < machine DRAM, but
    - chunk size shouldn't be too small (~1 GB is OK)

Q: *Do you tune any of these when using an RDBMS? :)*

Dask still lacks "physical data independence"!

https://docs.dask.org/en/latest/best-practices.html

Dask: Task-Parallelism Best Practices

- **Use the Diagnostics dashboard:**
  - Monitor # tasks, core/node usage, task completion
- **Task Graph sizes:**
  - Too large: Bottlenecks
    (serialization / communication / scheduling)
  - Too small: Under-utilization of cores/nodes
  - Rough guidelines:
    - Tune data chunk size to adjust # tasks (see previous point)
    - Break up a task/computation
    - Fuse tasks/computations aka “batching”

https://docs.dask.org/en/latest/best-practices.html
https://docs.dask.org/en/latest/diagnostics-distributed.html

Execution Optimization Tradeoffs

- Be judicious in tuning data chunk sizes
- Be judicious in batching vs breaking up tasks

Speedup is a function of the above factors.